Eval-Driven Development for AI Employees

Mode 2 Phase 3 (Scale the Workforce), Chapter 6/9 — Revision & Viva-Prep Cheatsheet (English + Roman Urdu)
Source: agentfactory.panaversity.org/docs/eval-driven-development-crash-course

EN: Test-Driven Development gave confidence in CODE (deterministic). Eval-Driven Development (EDD) gives confidence in AGENT BEHAVIOR (probabilistic). '10 years from now, shipping an agent without an eval suite will look like shipping SaaS without unit tests today.'
Roman Urdu: Test-Driven Development ne CODE (deterministic) mein confidence di. Eval-Driven Development (EDD) AGENT BEHAVIOR (probabilistic) mein confidence deta. '10 saal baad, eval suite ke bina agent ship karna aisa lagega jaisa aaj unit tests ke bina SaaS ship karna.'

1. Why Traditional Tests Aren't Enough

- Agents are hard in 5 ways: probabilistic, multi-step, tool-using, context-sensitive, connected to external systems. A full traditional test suite catches only ~10-30% of production failures.
- **3 levels of 'behavior':** Level 1 Final output, Level 2 Tool-use record, Level 3 Full trace -- grading only the final answer is like grading an exam from its last paragraph.

2. The Evaluation Pyramid (9 Layers)

- Foundation (unit, integration) -> LLM/Agent evaluation (output, tool-use, trace, RAG/knowledge) -> Operational reliability (safety/policy, regression, production). Each layer catches failures invisible to the layers below it.
- Output evals catch format violations, refusals, factual errors, hallucinations -- they systematically MISS process failures that happened to produce a correct output.
- RAG evals (Ragas, 5 metrics): Context Relevance, Faithfulness, Answer Correctness, Context Recall, Context Precision.

3. The Stack + the Golden Dataset

- 4-tool stack: OpenAI Agent Evals (+trace grading), DeepEval ('pytest for agent behavior', CI), Ragas (RAG-specific), Phoenix (production observability).
- **Golden dataset is the MOST undervalued artifact** -- quality order: representativeness (70% typical + 30% edge cases), difficulty stratification, ground-truth quality, source diversity, version control.
- 5 failure patterns: Imagination Trap, Easy-Mode Bias (false confidence), Single-Author Problem, Stale-Dataset Problem, Pass-Threshold Inflation.

4. What Evals Can't Measure + Closing

- Evals catch pattern-matching, drift, named safety bounds, tool-use correctness well. They're LIMITED on: novel situations, value alignment at edge cases, subjective quality, adversarial behavior.
- **5 anti-patterns:** calling output-only evals 'safe', using LLM-as-judge without calibration, building a big dataset without understanding failure categories, mistaking dashboards for evals, only running evals before launch.

Key Terms Cheatsheet

One line each — English + Roman Urdu.

Term	English	Roman Urdu
Eval-Driven Development (EDD)	The agentic-AI version of TDD -- confidence in probabilistic BEHAVIOR, not deterministic code.	Agentic-AI version of TDD -- probabilistic BEHAVIOR mein confidence, deterministic code nahi.
9-Layer Evaluation Pyramid	Foundation, LLM/Agent evaluation, Operational reliability -- layered, not redundant.	Foundation, LLM/Agent evaluation, Operational reliability -- layered, redundant nahi.
Golden dataset	The most undervalued artifact -- bad dataset means bad evals, no matter the tooling.	Sabse undervalued artifact -- bad dataset ka matlab bad evals, tooling kuch bhi ho.
Trace eval	Checks the full execution path, catching reasoning/handoff/guardrail-bypass failures.	Poora execution path check karta, reasoning/handoff/guardrail-bypass failures pakarta.
Eval-improvement loop	Define task -> run -> capture trace -> grade -> identify failure mode -> improve -> rerun.	Define task -> run -> capture trace -> grade -> identify failure mode -> improve -> rerun.

Quick Self-Check

Question	Answer
Why can an agent pass all traditional tests but still fail in production?	Each component can be individually correct while the overall reasoning path still produces a wrong result -- like the Maya example, refunding the wrong customer with a similar email.
Why is grading only the final output not enough?	It's like grading an exam from its final paragraph -- it misses process failures, lucky correctness, and reasoning errors hidden by post-hoc rationalization.
What's the single most undervalued artifact in this discipline?	The golden dataset -- a bad dataset guarantees bad evals no matter how good the tooling is.
Why must the LLM-as-judge model be different from the model running the agent?	To avoid self-grading bias -- a model is more likely to rate its own outputs favorably than an independent judge would.

Where This Leads / Sources

Leads to: Deploy Your Agent Harness, Choosing Agentic Architectures. Full 7-Decision hands-on lab (golden dataset -> output -> trace -> tool-use/safety -> RAG -> CI/CD -> Phoenix) in Parts 4-5.

Source: docs/eval-driven-development-crash-course/ (this repo), content originally via Zia Tutor AI connector.